CSE552 Program Analysis
Assignment 4: Interprocedural Interval Analysis

1 Overview

In this assignment, you will implement interprocedural, context-insensitive interval analysis for a subset of the Rust MIR.

Input. A Rust crate containing one or more functions. The entry point is a function named `main`.

Output. Your implementation should compute, for each MIR location of each analyzed function, an abstract state mapping each local variable to an interval.

2 Getting the Code and Running

2.1 Clone

```
git clone https://github.com/plrg-unist/cse552-a4.git
cd cse552-a4
```

2.2 Testing

This assignment requires `rustup` (`https://rustup.rs/`).

```
cargo test
```

- `cargo test` runs the tests.

3 What You Need to Read

You only need these files:

- `src/analysis.rs`
- `src/domains.rs`
- `src/tests.rs`

  You do **not** need to read other files to complete the assignment.

4 What You May Change

- You are allowed to change **only** `src/analysis.rs`.
- You may remove or replace the template code in `src/analysis.rs`.
- You must preserve the signature of the public function `analyze`.
- The only submission file is `src/analysis.rs`.

5 Analysis Requirements

Your implementation must satisfy the following requirements:

- Implement an interprocedural, context-insensitive interval analysis.
- Use `main` as the entry point.
- Use widening and narrowing.
- Apply widening at **every** node, not only at loop heads.
- At the entry of `main`, all parameters of `main` must be initialized to top, and all other locals must be bottom.
- Abstract operations for `+`, `-`, `*` and `/` must be sound and achieve the best possible precision for the interval domain.
- Abstract filtering for `<`, `<=`, `>`, `>=`, `==` and `!=` must be sound and achieve the best possible precision for the interval domain.
- Assume that no arithmetic overflow occurs.

5.1 Widening and Narrowing

Use the following interval widening operator ∇:

$$\bot \nabla y = y \qquad x \nabla \bot = x$$

and for non-bottom intervals,

$$[l_1, h_1] \nabla [l_2, h_2] = [l_3, h_3]$$

where

$$l_3 = \begin{cases} l_1 & \text{if } l_1 \leq l_2 \\ -\infty & \text{otherwise} \end{cases} \qquad h_3 = \begin{cases} h_1 & \text{if } h_1 \geq h_2 \\ \infty & \text{otherwise} \end{cases}$$

Use the following interval narrowing operator $\triangle$:

$$\bot \triangle y = \bot \qquad x \triangle \bot = \bot$$

and for non-bottom intervals,

$$[l_1, h_1] \triangle [l_2, h_2] = [l_3, h_3]$$

where

$$l_3 = \begin{cases} l_2 & \text{if } l_1 = -\infty \\ l_1 & \text{otherwise} \end{cases} \qquad h_3 = \begin{cases} h_2 & \text{if } h_1 = \infty \\ h_1 & \text{otherwise} \end{cases}$$

6 Supported MIR Subset

Your implementation only needs to handle the MIR constructs described below.

6.1 Function-Level Assumptions

- The entry point is named `main`.
- Each function, including `main`, may have zero or more parameters.
- Each parameter has type `i32`.
- The return type of every function is `i32`.

6.2 Statements, Rvalues, and Terminators

- `Assign` is the only possible `StatementKind`.
- Only integers (`i32`) are used.
- `Place` has no projections; only plain `Local`s appear.
- `Use` and `BinaryOp` are the only possible `Rvalue` variants.
- The only possible arithmetic `BinOp` variants are `Add`, `Sub`, `Mul`, and `Div`.
- The only possible comparison operators are `Lt`, `Le`, `Gt`, `Ge`, `Eq`, and `Ne`.
- `Goto`, `SwitchInt`, `Return`, and `Call` are the only possible `TerminatorKind` variants.
- Each function has exactly one `Return`.
- Each `SwitchInt` has exactly two branches: the true branch (value `1`) and the false branch (the default branch).
- Function pointers are not used, and function calls are direct calls to known functions.

6.3 Conditional Branch Assumption

Each `SwitchInt` follows this pattern:

- Its discriminant is a `Local`.
- That local is assigned exactly once, immediately before the switch.
- The assignment has the form `b = x cop n`, where `x` is a `Local`, `n` is an `i32` literal, and `cop` is one of `Lt`, `Le`, `Gt`, `Ge`, `Eq`, or `Ne`.
- You are not expected to track the value of `b` in the abstract state.
- Instead, use the underlying comparison directly when transferring through `SwitchInt`.

7 Testing and Grading

7.1 Grading Policy

- Your score is proportional to **hidden** test pass rate.
- The test cases will not intentionally test various edge cases related to abstract operations and filtering, as those are mainly the scope of the previous assignment.
- Late submission is prohibited.

7.2 Academic Integrity and LLM Policy

- You may reuse your own solutions to previous assignments.
- Do not get help from other people for solving this assignment.
- Do not give your solution to other people.
- Do not upload your solution to any public repository.
- Any cheating results in an **F** grade.
- You are allowed to use LLM tools, and using them is highly recommended.

8 Recommendation

Add your own tests to `src/tests.rs`. This is strongly recommended because only a few test cases are provided. The test cases use Rust's custom MIR to define functions with explicit MIR-level control flow. This section explains how to write your own.

8.1 Test Structure

Each test defines a list of functions and calls a helper of the following shape:

```
test(funcs, expected)
```

where:

- `funcs` is a slice of `(name, params, body)` tuples.
- `name` is the function name, such as `"main"` or `"foo"`.
- `params` is the parameter list of the function (e.g., `"x: i32, y: i32"` or `""` for no parameters).
- `body` is the custom MIR body.
- `expected` is a slice of `(func, bb, stmt_idx, state)` tuples.

8.2 Local Numbering

Locals are numbered separately for each function in this order:

1. `_0`: the return place (`RET` in custom MIR).
2. `_1`, `_2`, ...: function parameters, in declaration order.
3. Next available indices: temporaries declared with `let` at the top, in declaration order.

Example. For `fn foo(x: i32, y: i32) -> i32` with `let a; let b;`: `_0`=RET, `_1`=x, `_2`=y, `_3`=a, `_4`=b.

8.3 Writing the MIR Body

Variable declarations. All `let` declarations must appear at the top, before any basic block:

```
let a;
let b;
```

Basic blocks. The first (entry) block uses bare braces `{ }` with no label. Subsequent blocks use `label = { }`:

```
{
    a = 0;
    Goto(bb1)
}
bb1 = {
    RET = a;
    Return()
}
```

Statements.

- `x = y` — `Assign` with `Rvalue::Use`
- `x = y + z` — `Assign` with `Rvalue::BinaryOp`
- Operands can be locals or integer literals (e.g., `x = y + 1`)

Terminators. Every block must end with exactly one terminator:

- `Return()` — return from the function
- `Goto(bb1)` — unconditional jump
- `match var { true => bb1, _ => bb2 }` — boolean branch; this lowers to `SwitchInt`
- `Call(x = foo(a, b), ReturnTo(bb1), UnwindContinue())` — function call; this calls `foo`, stores its return value in `x`, and continues at `bb1`.